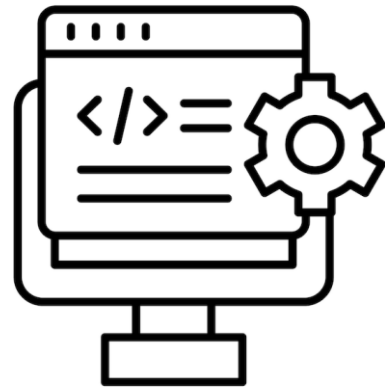


INT 161

Basic Backend Development (การพัฒนาแบ็กเอนด์พื้นฐาน)



RESTful API Development

Course Description

ความรู้เบื้องต้นเกี่ยวกับ RESTful API การเชื่อมต่อกับฐานข้อมูล API ระบุตัวตน และความปลอดภัย การเรียงลำดับ การแบ่งข้อมูลออกเป็นหน้า และการกรองข้อมูล

Basic RESTful API, Connection with Database, API Authentication and Security, Sorting, Pagination and Filtering

PLO: 2C-I; 2D-E; 2F-E; 5BE-I; 5FS-I

2C ออกแบบองค์ประกอบทางด้านเทคโนโลยีสารสนเทศที่สนองความต้องการทางธุรกิจได้

2D พัฒนากองประกอบทางด้านเทคโนโลยีสารสนเทศตามที่ได้กำหนดไว้ได้

2F ทดสอบองค์ประกอบและการเชื่อมต่อขององค์ประกอบทางด้านเทคโนโลยีสารสนเทศได้อย่างครบถ้วนบนข้อจำกัดที่มีได้

5BE ออกแบบและพัฒนาเว็บแอปพลิเคชันส่วนแบ็กเอนด์ที่ใช้ในระดับองค์กรได้

5FS ออกแบบและพัฒนาเว็บแอปพลิเคชันที่ใช้ในระดับองค์กร ทั้งในส่วนฟรอนเอนด์ และแบ็กเอนด์ได้

PLO แบ่งเป็น 5 ระดับ คือ

1. Introduce (I) – ระดับแนะนำ

2. Expand (E) – ต่อยอดจากองค์ความรู้/ทักษะ เดิมที่ได้พัฒนามาก่อนหน้า (INT141, INT150)

3. Strengthen (S) – มีความชำนาญมากขึ้น

4. Competence Level 1 (C1) – สมรรถนะในเชิงกว้างที่เป็นที่คาดหวังของบัณฑิตของหลักสูตร

5. Competence Level 2 (C2) – ผู้เรียนแสดงสมรรถนะในเชิงลึกที่เฉพาะเจาะจง ในสายอาชีพที่ผู้เรียนเลือก

Evaluation & Resources

2nd Exam. (Theory) 30%

- Multiple choices, single choose
- Describe

2nd Exam. (Practice) 20%

- Write a program to solve the problem.

3rd Exam. (Theory) 30%

- Multiple choices, single choose

3rd Exam. (Practice) 20%

- Write a program to solve the problem.

Resources:

- <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>
- <https://www.w3schools.com/nodejs/default.asp>
- <https://www.tutorialspoint.com/nodejs/index.htm>
- <https://restfulapi.net>
- <https://restful-api.dev>

Course outline – Group B (Thursday)

No.	Date	Topic
1	6 Aug 2026	Introduction
2	13 Aug 2026	Node.js vs Bun
3	20 Aug 2026	RESTful API
4	27 Aug 2026	Express framework
5	3 Sep 2026	Connection to Database
	7 -11 Sep 2026	1 st Examination
6	17 Sep 2026	Relationship Modeling/ORM
7	24 Sep 2026	Prisma ORM
8	1 Oct 2026	Validation & Error Handling
9	8 Oct 2026	Data Transfer Object (DTO)
10	15 Oct 2026	Filtering Sorting & Pagination
	19-26 Oct 2026	2 nd Examination

No.	Date	Topic
11	5 Nov 2026	JWT : Jason Web Tokens
12	12 Nov 2026	Authentication/Authorization
13	19 Nov 2026	Integrated Project Support Topic
14	26 Nov 2026	Integrated Project Support Topic
	30-11 Dec 2026	3 rd Examination

Course outline – Group A (Friday)

No.	Date	Topic
1	7 Aug 2026	Introduction
2	14 Aug 2026	Node.js vs Bun
3	21 Aug 2026	RESTful API
4	28 Aug 2026	Express framework
5	4 Sep 2026	Connection to Database
	7 -11 Sep 2026	1 st Examination
6	18 Sep 2026	Relationship Modeling/ORM
7	25 Sep 2026	Prisma ORM
8	2 Oct 2026	Validation & Error Handling
9	9 Oct 2026	Data Transfer Object (DTO)
10	16 Oct 2026	Filtering Sorting & Pagination
	19-26 Oct 2026	2 nd Examination

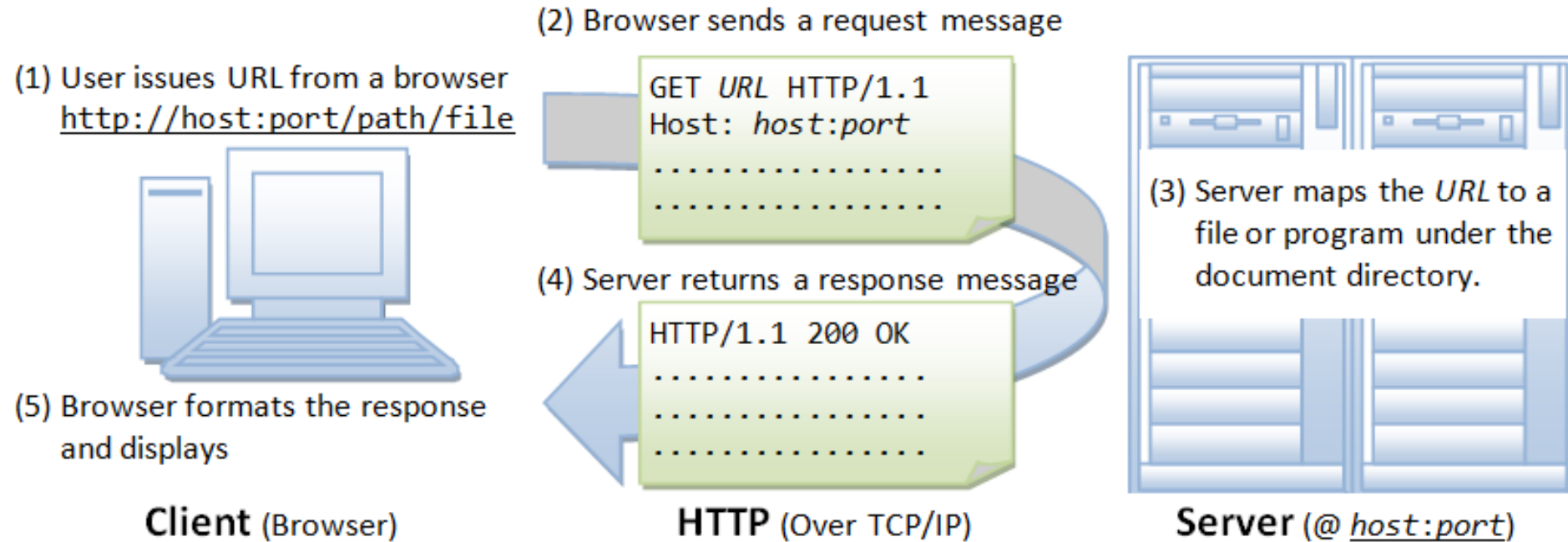
No.	Date	Topic
11	6 Nov 2026	JWT : Jason Web Tokens
12	13 Nov 2026	Authentication/Authorization
13	20 Nov 2026	Integrated Project Support Topic
14	27 Nov 2026	Integrated Project Support Topic
	30-11 Dec 2026	3 rd Examination



Introduction to Web Application

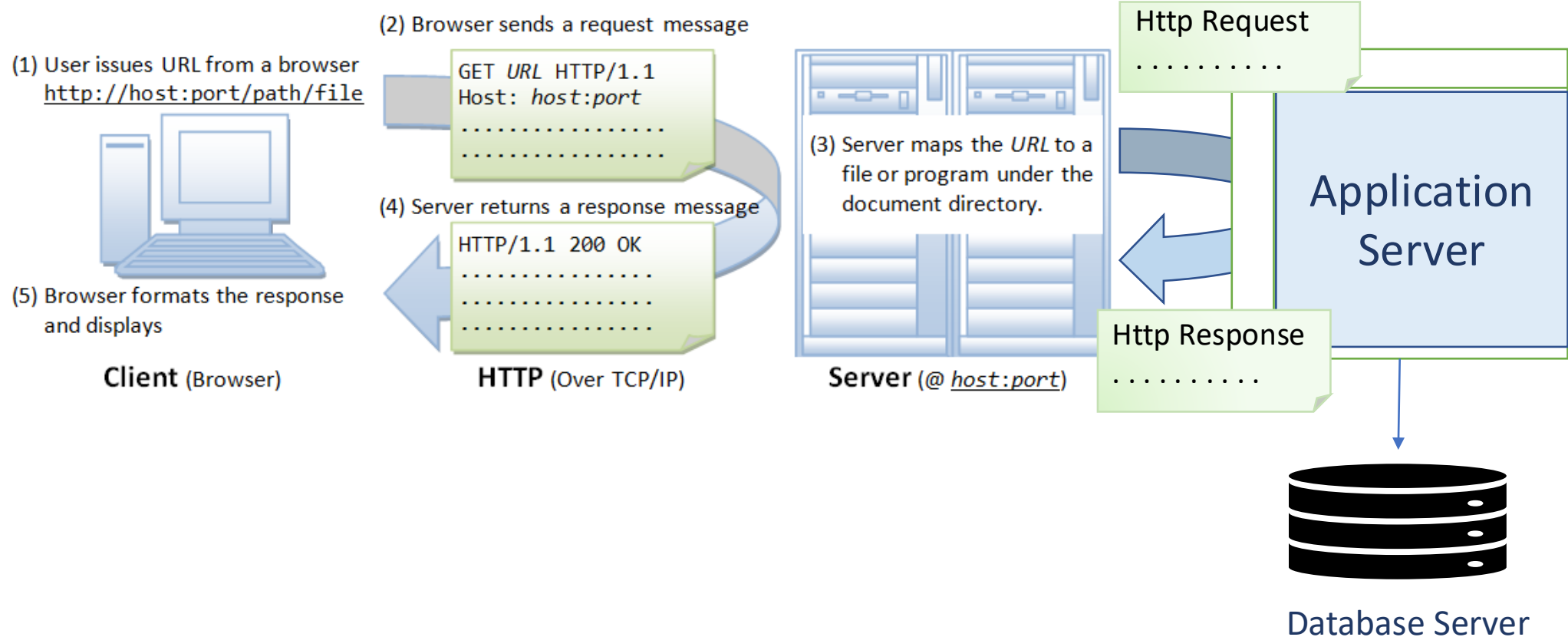
pichet@sit.kmutt.ac.th

HTTP Protocol



https://www.w3schools.com/whatis/whatis_http.asp

Web Application



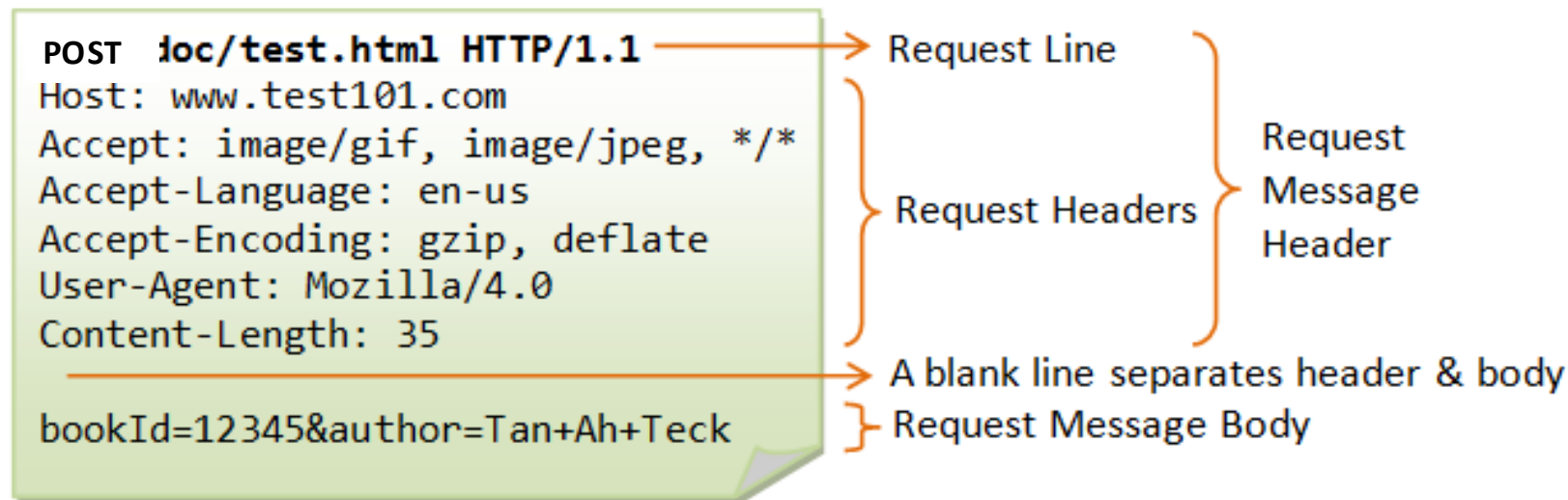
World Wide Web Communication

- **HTTP** stands for **H**yper **T**ext **T**ransfer **P**rotocol
- **WWW** is about communication between web **clients** and **servers**
- Communication between client computers and web servers is done by sending **HTTP Requests** and receiving **HTTP Responses**
- **Clients** are often browsers (Chrome, Edge, Safari), but they can be any type of program or device.
- **Servers** are most often computers in the cloud.

HTTP Request Structure

- There are three locations where you can store and send information within a request, each with its own abilities and restrictions. Here is a list of possible locations:
 - HTTP Headers
 - HTTP Methods
 - HTTP Bodies

http://demo.codingnomads.co:8080/tasks_api/tasks



HTTP Response Structure

- Every HTTP response follows a structured layout consisting of three main parts:
 - The Status Line (First Line)
 - Http Header
 - The Message Body (Payload)

```
HTTP/1.1 200 OK
```

```
Date: Wed, 05 Aug 2026 20:30:00 GMT
```

```
Server: Apache/2.4.41 (Unix)
```

```
Content-Type: text/html; charset=UTF-8
```

```
Content-Length: 138
```

```
Connection: close
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Hello World</title>
```

```
</head>
```

```
<body>
```

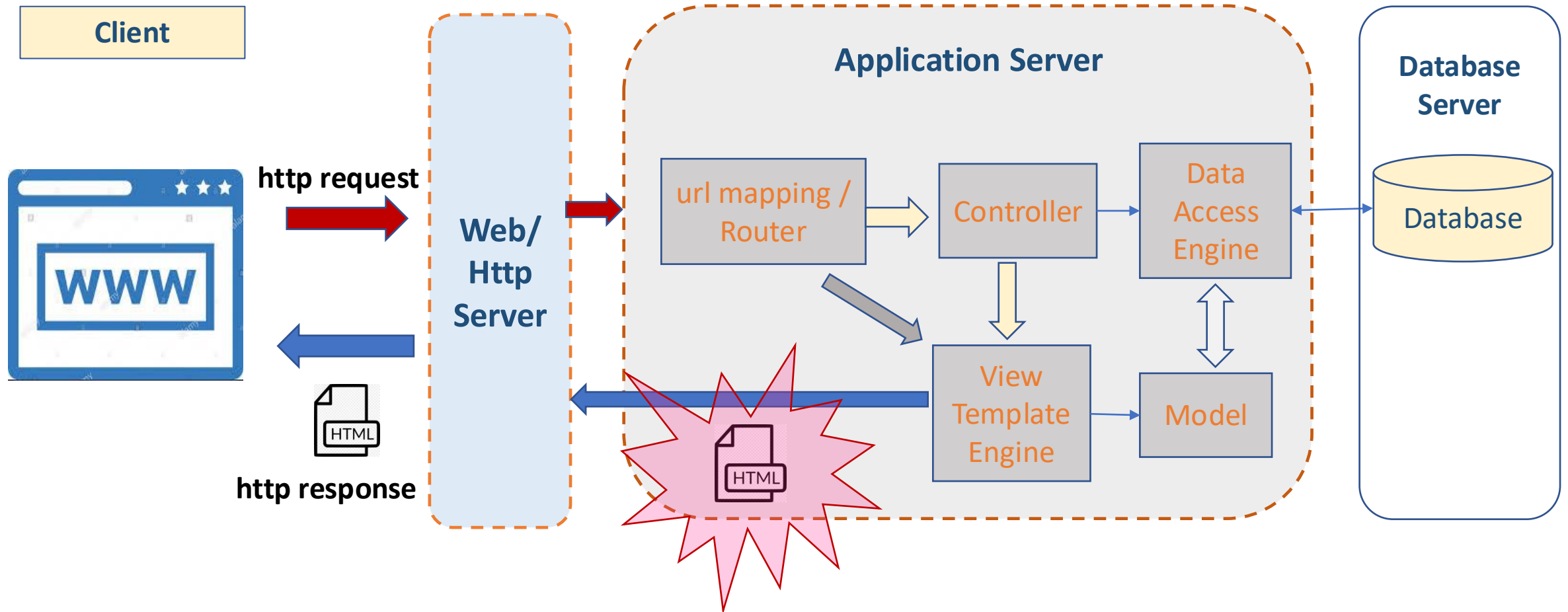
```
    <h1>Welcome to my website!</h1>
```

```
</body>
```

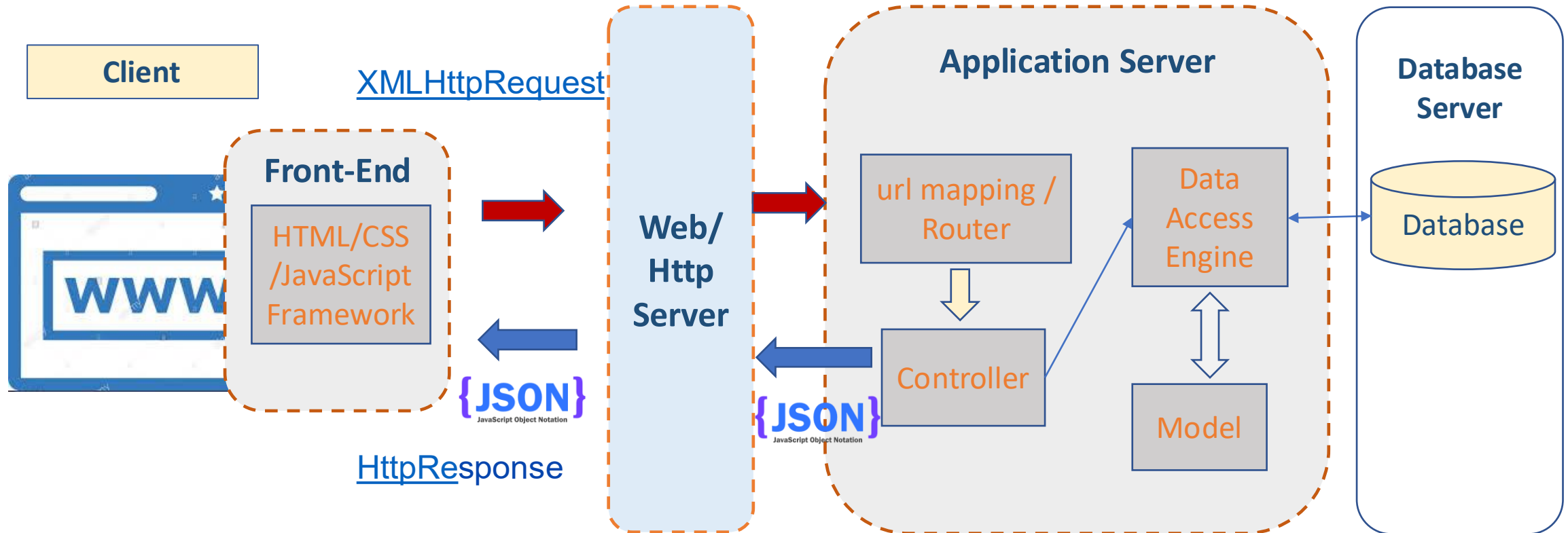
```
</html>
```

A Multi-Page Application (MPA) Web Application

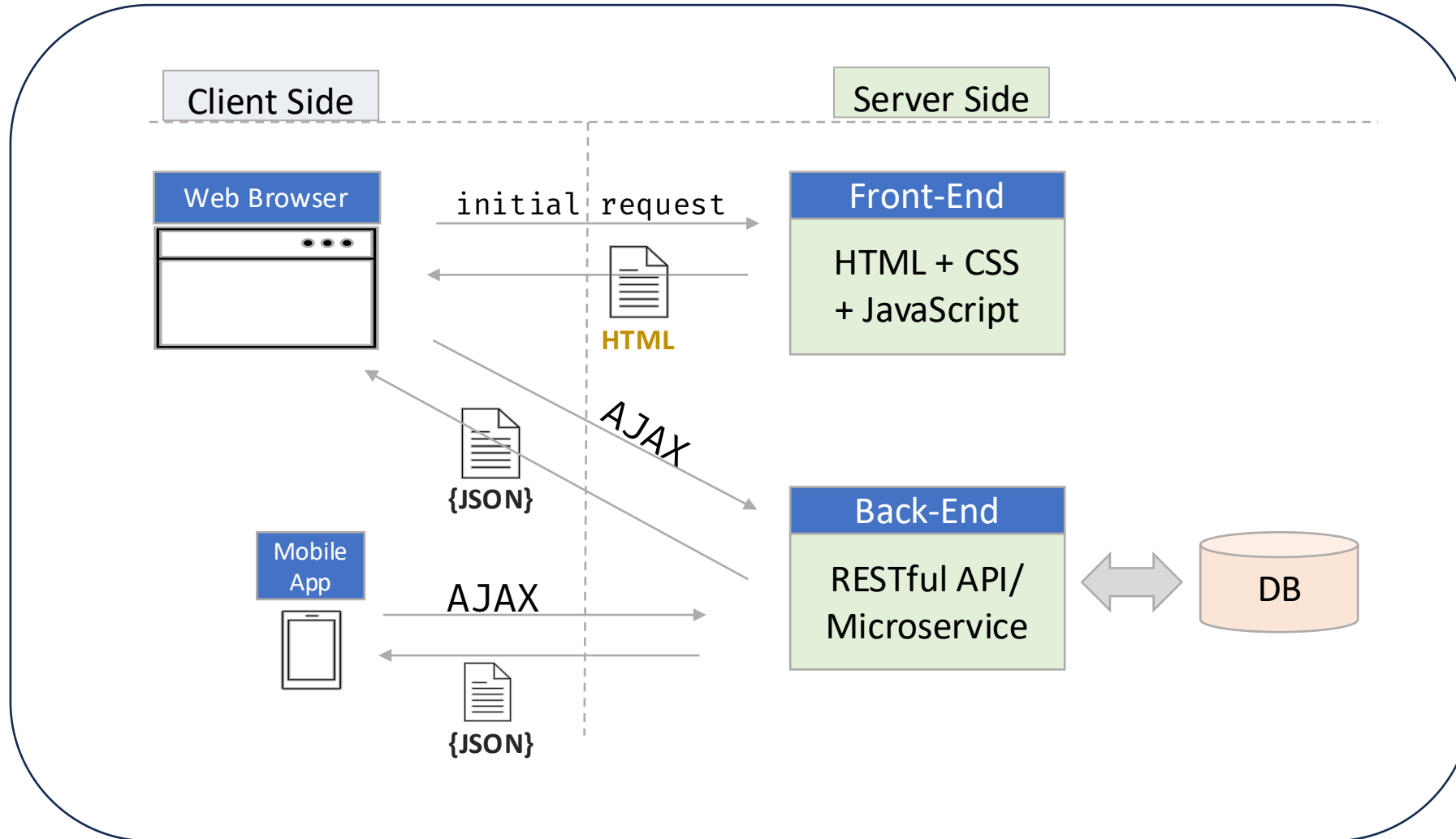
is a traditional web application architecture where each new page or view is loaded from the server with a full page refresh.



MVC Web Application Architectures (modern/spa)



SPA – Single Page Application architectures



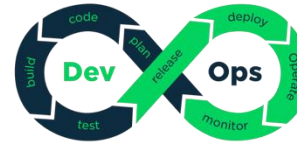
JSON : JavaScript Object Notation

- A lightweight data-interchange format.
- Easy for humans to read and write.
- Easy for machines to parse and generate.
- Based on a subset of the JavaScript Programming Language Standard ECMA-262 3rd Edition - December 1999.
- JSON is a text format that is completely language independent but uses conventions that are familiar to programmers of the C-family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others.

<https://www.json.org/json-en.html>

<https://education.launchcode.org/js-independent-track/chapters/fetch-json/index.html>

Full Stack Developer



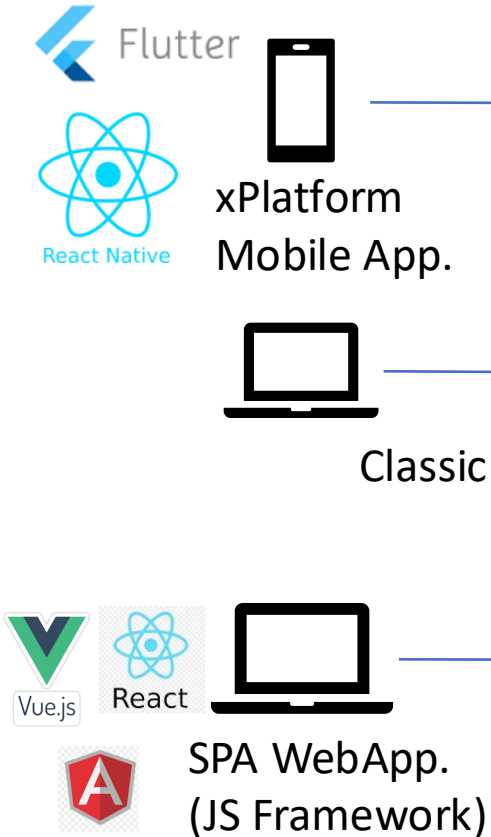
Client Side

Server Side

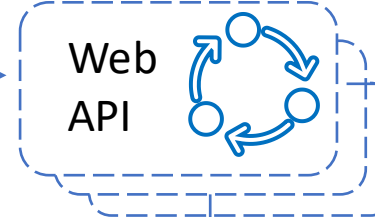
{ REST }

NoSQL

SQL



Microservice



Database



Database



EIS: Enterprise Info. System



Full Stack Web Developer

- A full stack web developer is a person who can develop both client and server software.
 - Program a browser (like using JavaScript, jQuery, Angular, or Vue)
 - Program a server (like using Java, PHP, ASP, Python, or NodeJS)
 - Program a database (like using SQL, SQLite, or MongoDB)



https://www.w3schools.com/whatis/whatis_fullstack.asp

Web Development Roadmaps (1)

- Front-End Roadmap

- Every Web Developer must have a basic understanding of HTML, CSS, and JavaScript.
- Responsive Web Design is used in all types of modern web development.
- ECMAScript 5 (JavaScript 5) is supported in all modern browsers.
- On the CSS side you should choose a framework for responsive web design:
 - Bootstrap / Material Design / W3.CSS / Tailwind
- On the JavaScript side you should learn at least one modern framework:
 - React.js / Angular.js / **Vue.js** / W3.JS



<https://www.w3schools.com/whatis/default.asp>

Web Development Roadmaps (2)

- Back-End Roadmaps
 - SQL / NoSQL
 - Language: Java / PHP / ASP, C# / Python / Node.js
 - Framework: Spring Boot / Laravel / .NET / Django / Adonis, Express+Prisma
 - Web Service
 - Microservice



https://www.w3schools.com/whatis/whatis_fullstack.asp

Web Services

- Types of Web services are classified into two types
- SOAP Web Services
 - SOAP is an abbreviation for **Simple Object Access Protocol**. It establishes the standard XML format. It also outlines how web services are built. We utilize Web Service Definition Language (WSDL) to describe the request and response XML formats
- RESTful Web Services
 - It was created by [Roy Thomas Fielding, who also created HTTP](#). The primary purpose of RESTful web services is to improve the efficiency of online services. RESTful web services attempt to describe services by utilizing the many principles currently existing in HTTP. REST is a design approach, not a protocol.

INT 161

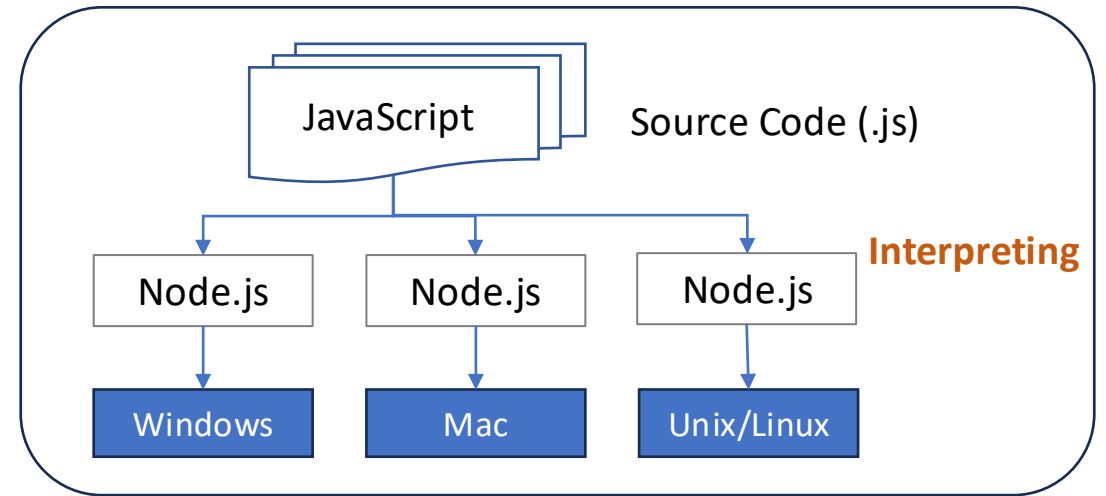
Introduction to Node.js



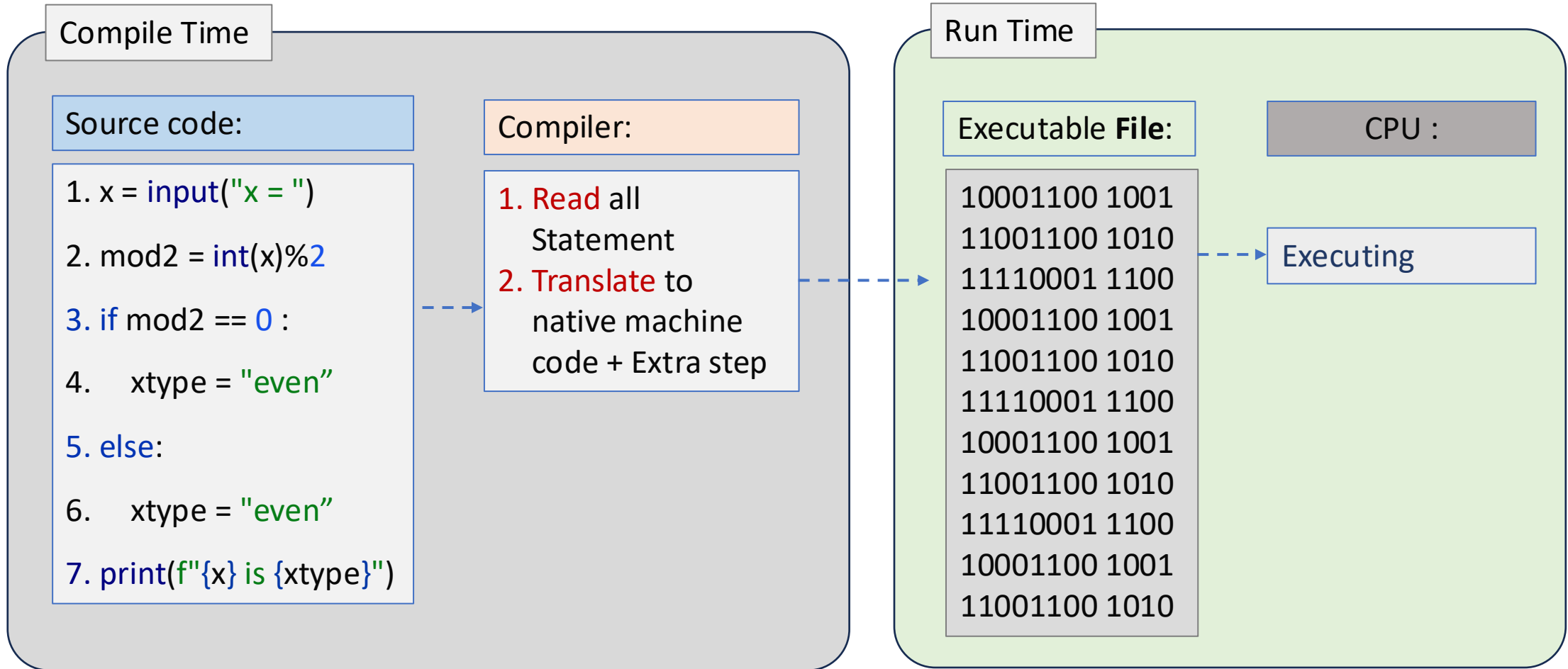
Understanding the Basics of node.js

What is Node.js?

- Node.js is not a programming language
- Node.js is a runtime that converts JavaScript code into machine code.
- Node.js is an open source server environment
- Node.js is free
- Node.js runs on various platforms (Windows, Linux, Unix, Mac OS, etc.)
- Node.js uses **JavaScript** on the server

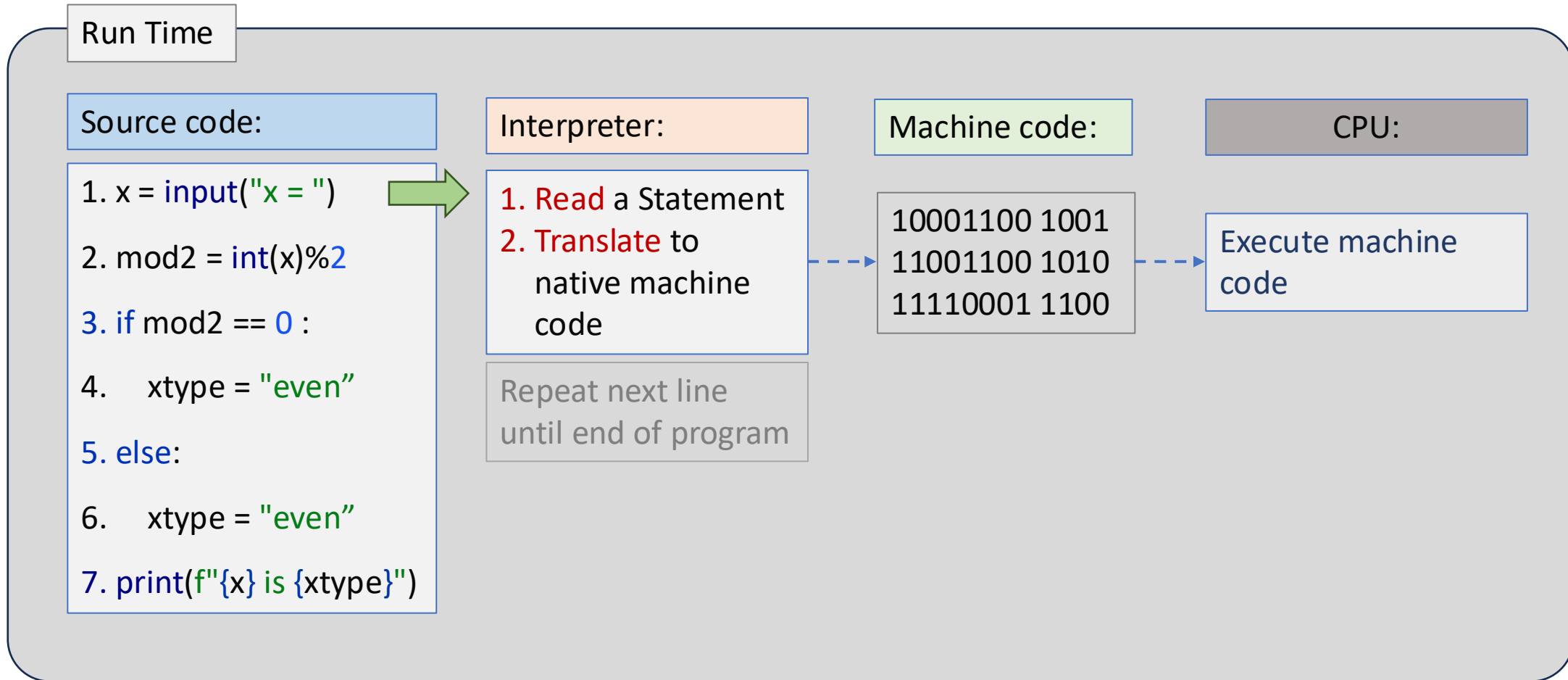


Computer Language Translator: **Compilation**



C, C++, Go

Computer Language Translator: Interpreter



Compilation vs Interpretation

1. Translation Timing and Workflow

- **Compilation:** The compiler checks the whole program for errors and translates it into a binary executable entirely upfront. This adds an extra step during development, but ensures the code is ready to run independently.
- **Interpretation:** The interpreter translates code on the fly during runtime, executing each line immediately. This enables rapid development and easy debugging, as you can run the code without waiting for a compilation step.

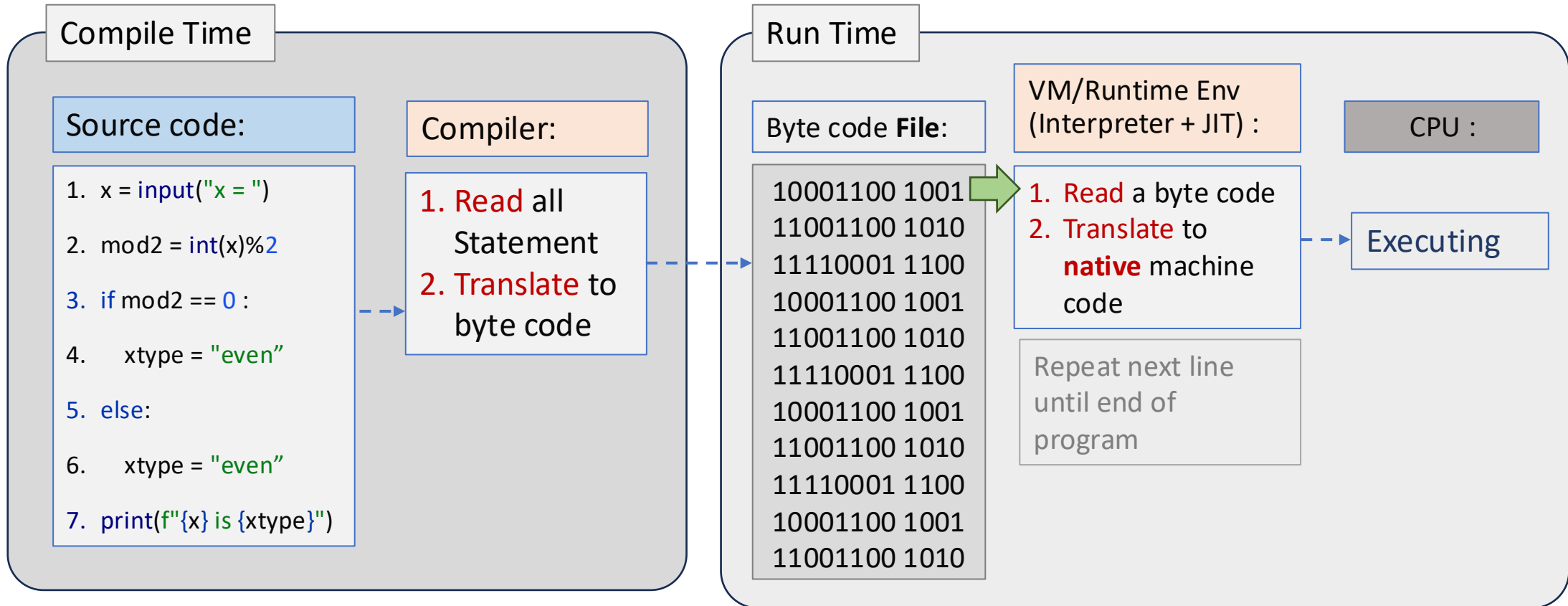
2. Performance and Speed

- **Compilation:** Yields faster execution times because the CPU can directly execute the highly optimized machine code.
- **Interpretation:** Generally slower because the translation process happens concurrently with the execution, adding overhead every time the program runs.

3. Portability vs. Privacy

- **Compilation:** Compiled executables are usually built for a specific operating system and processor architecture, making them less portable. However, they keep the original source code private because you only distribute the final binary file.
- **Interpretation:** Offers excellent cross-platform portability - you can run the same source code on any machine that has the correct interpreter installed. The downside is that your source code must be distributed, making it publicly visible.

Computer Language Translator: **Hybrid**



Java, Kotlin, Groovy, C# (.NET), ...

Why Learn Node.js?

Node.js can be used to full fill multiple purpose like server-side programming, build APIs, etc.

- Node.js is used for server-side programming with JavaScript. Hence, you can use a single programming language (JavaScript) for both front-end and back-end development.
- Node.js **implements asynchronous execution** of tasks in a single thread with async and await technique. This makes Node.js application significantly faster than multi-threaded applications.
- Node.js is being used to build command line applications, web applications, real-time chat applications, REST APIs etc.

How to Install Node.js?

Download and install nvm:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.2/install.sh | bash
```

in lieu of restarting the shell

```
\. "$HOME/.nvm/nvm.sh"
```

Download and install Node.js:

```
nvm install 22
```

Verify the Node.js version:

```
node -v          # Should print "v22.14.0"
```

```
nvm current      # Should print "v22.14.0"
```

Verify npm version:

```
npm -v           # Should print "10.9.2"
```

The official Node.js website has installation instructions for Node.js: <https://nodejs.org>

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Download Node.js (LTS) 

Downloads Node.js **v22.14.0**¹ with long-term support.
Node.js can also be installed via [version managers](#).

Want new features sooner? Get **Node.js v23.11.0**  instead.

What is a Node.js File?

- Node.js files contain code that runs on the server.
- They usually have the .js extension (.ts for typescript) and can be started with the node command.
 - Node.js files run tasks when certain events happen (like a web request)
 - They must be started on the server to have any effect
 - They use JavaScript syntax
- Example: Running a Node.js File
 - > `node app.js`

myfirst.js

```
> node myfirst.js
```

```
const readline = require('node:readline/promises');
const { stdin: input, stdout: output } = require('node:process');

async function askQuestion() {
  const rl = readline.createInterface({ input, output });

  try {
    const name = await rl.question('What is your name? ');
    const birthyear = await rl.question('What is your birth year in B.E.? ');

    const currentYear = new Date().getFullYear();
    const age = currentYear - birthyear + 543;

    console.log(`Thank you ${name}, aged ${age}!`);
  } finally {
    rl.close();    // Ensures the terminal stream is closed
  }
}

askQuestion();
```

Creating Node.js Web Application

- To create a "Hello, World!" web application using Node.js, you need the following three important components
 - Import required modules – We use the require directive to load Node.js modules.
 - Create server – A HTTP Server which will listen to clients.
 - Read request and return response – The server created in an earlier step will read the HTTP request made by the client which can be a browser or a console and return the response.

my-first-app.js

```
> node my-first-app.js
```

```
http = require('node:http');
listener = function (request, response) {
  // Send the HTTP header
  // HTTP Status: 200 : OK
  // Content Type: text/html
  response.writeHead(200, {'Content-Type': 'text/html'});

  // Send the response body as "Hello World"
  response.end('<h2 style="text-align: center;">Hello World</h2>');
};

server = http.createServer(listener);
server.listen(3000);

// Console will print the message

console.log('Server running at http://127.0.0.1:3000/');
```


my-second-app.js

```
> node my-second-app.js
```

```
http = require('node:http');
listener = function (request, response) {
  response.write('rawHeaders: ' + request.rawHeaders + '\n-----\n\n');
  response.write('http version: ' + request.httpVersion + '\n');
  response.write('url: ' + request.url + '\n');
  response.write('method: ' + request.method + '\n');
  response.write('headers.user-agent: ' + request.headers["user-agent"] + '\n');
  response.end('headers.host: ' + request.headers.host + '\n');
};

server = http.createServer(listener);
server.listen(3000);

// Console will print the message

console.log('Server running at http://127.0.0.1:3000/');
```

my-third-app.js

```
> node my-third-app.js
```

```
http = require('node:http');
listener = function (request, response) {
  if (request.url === '/user'){
    response.writeHead(200, {'Content-Type': 'application/json'})
    response.end(JSON.stringify({ name : 'John Doe'}))
  } else if (request.url === '/') {
    response.writeHead(200)
    response.end('Hello World')
  } else {
    response.writeHead(404)
    response.end('<Error: Page Not Found')
  }
};
server = http.createServer(listener);
server.listen(3000);

console.log('Server running at http://127.0.0.1:3000/');
```